Enrollment number - 240840116004

Practical : 1 Study the data analytics lifecycle and perform data acquisitiopn from csv,excel and json files using pandas

step 1: import the pandaas library

```
# Pandas is a Python library used to work with data.
# It helps us read, organize, analyze, and process data easily.

import pandas as pd

# pd is the short name (alias) for the Pandas library.
```

step 2: create sample data

```
# Create sample data by storing example values in a dictionary.
data = {
    "ID": [101,102,103,104,105,106,107],   # Student ID numbers
    "Name": ["Amii","Krish","Parth","Feny","Himani","Kavya","Riken"],    # Student names
    "Age" :[19,20,22,19,19,20,20],       # Student ages
    "City": ["Navsari","Surat","Navsari","Bharuch","Valsad","Bardoli","Surat"],  # Cities
    "Marks" :[78,88,90,75,69,94,89]                 # Student marks

}
```

step 3 : create a data frame

```
# A DataFrame is a table with rows and columns used to store data.
# df is the variable that stores the DataFrame
df = pd.DataFrame(data)

# Display the DataFrame
df
```

	ID	Name	Age	City	Marks
0	101	Amii	19	Navsari	78
1	102	Krish	20	Surat	88
2	103	Parth	22	Navsari	90
3	104	Feny	19	Bharuch	75
4	105	Himani	19	Valsad	69
5	106	Kavya	20	Bardoli	94
6	107	Riken	20	Surat	89

step 4: Save data as a csv file

```
# create a csv file
df.to_csv("students.csv", index=False)

# Display a message to confirm that the CSV file was created successfully
print("csv file created successfully.")
```

```
csv file created successfully.
```

step 5 : save data as a excel file

```
# Save the DataFrame as an Excel file without including the row index
# index=False means the row numbers will not be saved in the file.
df.to_excel("students.xlsx",index=False)

# Display a message confirming that the Excel file was created successfully
print("Excel file created successfully.")
```

```
Excel file created successfully.
```

Step 6 : Save data as JSON file

```
# Save the DataFrame as a JSON file with each row stored as a separate record
# orient="records" saves each row as a separate JSON object in a list
df.to_json("students.json", orient = "records" , index=4)

# Display a message after the JSON file is created
print("JSON file created successfully.")
```

```
JSON file created successfully.
```

Step 7: Read the CSV file

```
# Read the data from the "students.csv" file and store it in a DataFrame
# pd.read_csv() -> Reads data from a CSV file
# "students.csv" -> Name of the CSV file to read
# df_csv -> Variable used to store the DataFrame

df_csv = pd.read_csv("students.csv")

# Display the DataFrame

df_csv
```

	ID	Name	Age	City	Marks
0	101	Amii	19	Navsari	78
1	102	Krish	20	Surat	88
2	103	Parth	22	Navsari	90
3	104	Feny	19	Bharuch	75
4	105	Himani	19	Valsad	69
5	106	Kavya	20	Bardoli	94
6	107	Riken	20	Surat	89

Step 8:Read the Excel file

```
# Read the data from the "students.xlsx" Excel file and store it in a DataFrame
# pd.read_excel() -> Reads data from an Excel file
# "students.xlsx" -> Name of the Excel file to read
# df_excel -> Variable used to store the DataFrame

df_excel = pd.read_excel("students.xlsx")

# Display the DataFrame

df_excel
```

	ID	Name	Age	City	Marks
0	101	Amii	19	Navsari	78
1	102	Krish	20	Surat	88
2	103	Parth	22	Navsari	90
3	104	Feny	19	Bharuch	75
4	105	Himani	19	Valsad	69
5	106	Kavya	20	Bardoli	94
6	107	Riken	20	Surat	89

Step 9: Read the JSON file

```
# Read the data from the "students.json" JSON file and store it in a DataFrame
# pd.read_json() -> Reads data from a JSON file
# "students.json" -> Name of the JSON file to read
# df_json -> Variable used to store the DataFrame

df_json = pd.read_json("students.json")

# Display the DataFrame
```

```
df_json
```

	ID	Name	Age	City	Marks
0	101	Amii	19	Navsari	78
1	102	Krish	20	Surat	88
2	103	Parth	22	Navsari	90
3	104	Feny	19	Bharuch	75
4	105	Himani	19	Valsad	69
5	106	Kavya	20	Bardoli	94
6	107	Riken	20	Surat	89

Step 10: Display the first five record

```
# Display the first 5 rows of the DataFrame
# .head() -> Displays the first 5 rows of the DataFrame (default)

df_csv.head()
```

	ID	Name	Age	City	Marks
0	101	Amii	19	Navsari	78
1	102	Krish	20	Surat	88
2	103	Parth	22	Navsari	90
3	104	Feny	19	Bharuch	75
4	105	Himani	19	Valsad	69

Step 11: Display the last five record

```
# Display the last 5 rows of the DataFrame
# .tail() -> Displays the last 5 rows of the DataFrame (default)
df_csv.tail()
```

	ID	Name	Age	City	Marks
2	103	Parth	22	Navsari	90
3	104	Feny	19	Bharuch	75
4	105	Himani	19	Valsad	69
5	106	Kavya	20	Bardoli	94
6	107	Riken	20	Surat	89

Step 12: Display dataset infromation

```
# Display information about the DataFrame, such as columns, data types, and missing values
# df_csv -> The DataFrame containing the CSV data

df_csv.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   ID      7 non-null      int64
 1   Name    7 non-null      object
 2   Age     7 non-null      int64
 3   City    7 non-null      object
 4   Marks   7 non-null      int64
dtypes: int64(3), object(2)
memory usage: 412.0+ bytes
```

Step 13: Display Statistical Summary

```
# Display summary statistics of the numerical columns in the DataFrame
# .describe() -> Shows summary statistics such as count, mean, standard deviation, minimum, maximum, and quartiles

df_csv.describe()
```

	ID	Age	Marks
count	7.000000	7.000000	7.000000
mean	104.000000	19.857143	83.285714
std	2.160247	1.069045	9.268482
min	101.000000	19.000000	69.000000
25%	102.500000	19.000000	76.500000
50%	104.000000	20.000000	88.000000
75%	105.500000	20.000000	89.500000
max	107.000000	22.000000	94.000000

Step 14 : Display dataset shape

```
# Display the number of rows and columns in the DataFrame
# .shape -> Returns the DataFrame's size as (rows, columns)

df_csv.shape
```

```
(7, 5)
```

Step 15 : Display column names

```
# Display the names of all columns in the DataFrame
# .columns -> Returns the names of all columns in the DataFrame

df_csv.columns
```

```
Index(['ID', 'Name', 'Age', 'City', 'Marks'], dtype='object')
```

Step 16: Display Data types

```
# Display the data type of each column in the DataFrame
# .dtypes -> Returns the data type of each column (e.g., int64, float64, object)

df_csv.dtypes
```

	0
ID	int64
Name	object
Age	int64
City	object
Marks	int64

dtype: object

Step 17: check the missing value

```
# Check the number of missing (null) values in each column
# .isnull() -> Checks for missing (null) values and returns True/False
# .sum() -> Counts the total number of missing values in each column

df_csv.isnull().sum()
```

	0
ID	0
Name	0
Age	0
City	0
Marks	0

dtype: int64

Step 18: Display a Single Column

```
# Display the "Marks" column from the DataFrame
# ["Marks"] -> Selects and displays only the "Marks" column

df_csv["Marks"]
```

	Marks
0	78
1	88
2	90
3	75
4	69
5	94
6	89

dtype: int64

Step 19: Display MUltiple Columns

```
# Display only the "Name" and "Marks" columns from the DataFrame
# [["Name", "Marks"]] -> Selects and displays only the "Name" and "Marks" columns

df_csv[["Name","Marks"]]
```

	Name	Marks
0	Amii	78
1	Krish	88
2	Parth	90
3	Feny	75
4	Himani	69
5	Kavya	94
6	Riken	89

Step 20: Filter Records

```
# Display only the rows where the Marks are greater than 75
# ["Marks"] -> Selects the "Marks" column
# > 75 -> Checks if the marks are greater than 75
# df_csv[...] -> Returns only the rows that meet the condition

df_csv[df_csv["Marks"] > 75]
```

	ID	Name	Age	City	Marks
0	101	Amii	19	Navsari	78
1	102	Krish	20	Surat	88
2	103	Parth	22	Navsari	90
5	106	Kavya	20	Bardoli	94
6	107	Riken	20	Surat	89

Step 21: Calculate Average Marks

```
# Calculate and display the average (mean) of the "Marks" column
# ["Marks"] -> Selects the "Marks" column
# .mean() -> Calculates the average value of the selected column

df_csv["Marks"].mean()
```

```
np.float64(83.28571428571429)
```

Step 22: Sort Data

```
# sort the dataset in descending order of marks.

df_csv.sort_values(by="Marks", ascending=False)

# sort_values() sort the dataset.
# by="Marks" -> Sorts the data using the "Marks" column
# ascending=False -> Sorts from highest to lowest values
```

	ID	Name	Age	City	Marks
5	106	Kavya	20	Bardoli	94
2	103	Parth	22	Navsari	90
6	107	Riken	20	Surat	89
1	102	Krish	20	Surat	88
0	101	Amii	19	Navsari	78
3	104	Feny	19	Bharuch	75
4	105	Himani	19	Valsad	69